TED64 — The Tiny EDitor, 64-bit Windows Edition

Version: 2.0 64-bit
Source: `TED64.ASM` (Microsoft Macro Assembler, x64; about 7,600 lines, 146 procedures)
Based on: TED 1.1 by Tom Kihlken, *PC Magazine*, 1988 (© Ziff Communications Co.)

TED64 is a small full-screen text editor for the Windows console. It began as a port of Tom Kihlken's 1988 DOS editor, TED, from 16-bit real-mode `.COM` code to a 64-bit Win32 console application. Versions 1.3 to 2.0 keep the original's key layout and much of its structure, but the core has been rebuilt:

- **Gap buffer**, so typing costs the same anywhere in a file of any size
- **Unicode (UTF-8)** text, with double-width CJK characters and emoji, and Unicode file names
- **Multi-level undo and redo** of every change
- **Find and Replace**, with Unicode case folding and regular expressions
- **Go to line**, a **configurable tab width**, and **automatic sideways scrolling**
- **Live window resizing**
- A **modified flag**, so the editor only asks questions when there are unsaved changes
- **Save without quitting** (F10)
- **16-bit screen and 32-bit line columns**, which remove the old 255-column limits
- **A screen of its own**, so your console contents come back untouched when you quit
- **Saves that keep the file's permissions and attributes**, and that won't overwrite read-only files
- **Careful with your files:** it won't open a locked or unreadable file as an empty one, warns before overwriting changes another program made, keeps hard links, ignores Ctrl+Break, and saves your text to a recovery file if the window is closed

The whole program is one assembly file with no runtime library. It links into a console `.exe` of about 31 KB that imports only from `kernel32.dll` and `user32.dll` (for the clipboard). Every procedure has Win64 unwind data, so debuggers can show the call stack.

---

1. Building

Command line (x64 Native Tools Command Prompt for VS)

```
ml64 TED64.ASM /link /subsystem:console /entry:main
```

Visual Studio IDE

1. Create an **Empty Project** (C++).
2. Right-click the project, choose **Build Dependencies → Build Customizations…**, and tick **masm**.

3. Add `TED64.ASM` to the project and set the platform to **x64**.
4. Set **Linker → System → SubSystem** to **Console**.
5. Optional: set **Linker → Advanced → Entry Point** to `main` so the C runtime isn't linked in.

The source names its import libraries itself (`includelib kernel32.lib` and `user32.lib`), so nothing needs adding to the link. It uses `OPTION CASEMAP:NONE`, so symbol names are case-sensitive.

Build-time options

Symbol	Default	Meaning
`TED_VERSION`	`"2.0 64-bit"`	Version text shown in the exit banner
`FILE_MAX`	`100000000h` (4 GB)	Largest text the editor will hold
`ADD_EOF_MARK`	`0`	`1` appends a DOS Ctrl-Z end-of-file byte on save
`MAX_COLS` / `MAX_ROWS`	1024 / 300	Largest console window the editor will use
`HIST_MAX`	`10000000h` (256 MB)	Space for each of the undo and redo histories
`LINE_MAX`	`10000h` (64 KB)	Bytes kept for F8/F9 and restored by F12
`FIND_MAX`	200	Longest search or replacement string, in bytes
`STEP_LIMIT`	100,000,000	Backtracking steps a regular-expression search may take
`GAP_MIN`	`10000h` (64 KB)	Smallest amount the gap grows by
`IO_CHUNK`	`40000000h` (1 GB)	Size of each `ReadFile` / `WriteFile` call
`RESERVE_SIZE`	`140040000h` (5 GB)	Address space first requested for each large buffer
`MIN_RESERVE`	`4000000h` (64 MB)	Smallest reservation tried before giving up

2. Using the editor

```
TED64 [/T:n] [filename]
```

If the file exists, it is loaded. If it doesn't exist (or its folder doesn't), you start with an empty buffer, and that name is offered when you save. Filenames that contain spaces must be quoted. Any Unicode file name works, both on the command line and at the "Save as:" prompt.

`/T:n` sets the tab width to *n* columns (1–32; the default is 8). `/Tn`, `-t:n` and `-tn` work too. A bad value prints the usage line and exits with code 1.

If the file exists but can't be opened, TED says why and stops, rather than showing an empty buffer that could later be saved over the real file:

Message	Cause
Can't open - the file is in use by another program	Another program has it locked
Can't open - access denied	You don't have permission to read it

Message	Cause
Can't open - that is a folder, not a file	The name is a folder
Can't open the file	Any other error

Files are opened allowing other programs to keep reading and writing them, so a log file that another program is still writing can be opened.

Exit codes: TED returns 0 after a normal quit, and 1 after an error ("Can't open...", "Read error", "File too big (the limit is 4 GB)", "Not enough memory", or console input being lost), so batch files and scripts can check `ERRORLEVEL`.

Ctrl+C and Ctrl+Break never end TED. Ctrl+C is COPY, and Ctrl+Break is ignored.

Closing the window (or logging off, or shutting Windows down) with unsaved changes writes the text to a recovery file, `name.SAV` next to the file (for example `notes.txt.SAV`), or `TED.SAV` in the current folder if the text has no name yet (always with CR/LF line endings). The file itself isn't saved, and nothing else is touched. The next time you open that file, the prompt line says "Unsaved text from a window that was closed is in notes.txt.SAV" until you press a key. TED never deletes a `.SAV` file; open it, copy what you need, and delete it yourself.

The bottom line of the window is the key prompt. Its last two columns show * when there are unsaved changes, and **I** (insert mode) or **O** (overstrike mode). In windows narrower than 88 columns the spaces between the keys are left out (the labels are in inverse video, so they still stand apart), and in very narrow windows the prompt is cut off on the right.

Function keys

Key	Command	What it does
F1	ABORT	Quits. If there are unsaved changes it first asks "Lose Changes (Y/N)?"
F2 / **Ctrl+Z**	UNDO	Takes back the last change (see "Undo and redo")
F3 / **Ctrl+C**	COPY	Copies the marked text to the clipboard and turns marking off
F4	MARK	Starts or cancels marking. Move the cursor to extend the selection
F5 / **Ctrl+X**	CUT	Removes the marked text to the clipboard
F6 / **Ctrl+V**	PASTE	Inserts the clipboard text at the cursor
F7 / **Ctrl+Q**	EXIT	Prompts "Save as:" with the filename (editable), saves and quits. **Esc** cancels. With no unsaved changes it just quits
F8	DEL EOL	Deletes from the cursor to the end of the line
F9	DEL L	Deletes the whole line
F10	SAVE	Prompts "Save as:" with the filename (editable), saves and carries on editing. **Esc** cancels. "Saved" appears on the prompt line until the next key

Key	Command	What it does
F11	—	Not used
F12	UDEL L	Undeletes the last line or end-of-line deleted by F8/F9

The clipboard

Cut and Copy put the text on the Windows clipboard, so you can paste it into other programs, and Paste takes whatever was copied last, in TED or in another program. Text TED copied pastes back byte for byte, even bytes that aren't valid UTF-8. Text from other programs has its line ends changed to CR/LF. If the clipboard is busy or holds no text, Paste uses TED's own copy.

Ctrl+V in Windows Terminal and the console. Windows Terminal, and the classic console with "Ctrl key shortcuts" turned on, keep Ctrl+V for themselves: they paste the clipboard by typing it into TED. Since TED's Copy and Cut fill the clipboard, the result is the same, with two differences. The text is inserted as if typed, so in overstrike mode it types over the text already there. And very long text is slower. Undo still takes back a whole console paste in one step, and the LF that some consoles send after each Enter is dropped. **F6** always pastes directly.

Undo and redo

Key	Action
F2 or **Ctrl+Z**	Undo the last change
Ctrl+Y	Redo the last change undone

Every change can be undone, back to the file as it was opened: typing, Delete, Backspace, Enter, overstrike, Cut, Paste, F8, F9, F12 and Replace. A run of typing counts as one step, ending at a space or Enter, so undo takes back a word at a time. A run of Delete or Backspace presses is also one step, and a Replace with **A** (all) is one step however many matches it changed. Undo puts the cursor where the change was and cancels any marking.

Making a new change after undoing clears the redo list. The * follows the history: undo back to the saved version and it disappears; redo and it comes back. Each history holds up to 256 MB (including the text of deletions). If a change won't fit, that history is emptied and editing carries on.

Find, Replace and Go to line

Key	Action
Ctrl+F or **Alt+F**	Find. Type the text (the last search is offered again), then **Enter**. **Esc** cancels
Ctrl+L	Find the next match of the last search
Ctrl+H	Replace: asks "Replace:" and "With:", then goes through the matches
Ctrl+R (in the Find or Replace prompt)	Switch between plain text and a regular expression; the prompt reads "Find (regex):" or "Replace (regex):"
Ctrl+G	Go to a line number

Find starts just after the cursor and wraps around to the top of the file. The match is shown, centred if it was off the screen. If nothing matches, "Not found" appears on the prompt line until the next key. A line number past the end of the file goes to the last line.

Case. Letters match in either case: A–Z, the Latin-1 and Latin Extended-A letters (É/é, Ł/ł, Ÿ/ÿ…), Greek (including accented letters and final sigma) and Cyrillic. Other characters must match exactly.

Regular expressions. A match never crosses a line end and is never empty.

Pattern	Matches
`.`	Any character
`[abc]`, `[a-z]`, `[^…]`	One character in the set, or not in it. `]` first, or `-` first or last, is literal. `\d`, `\w`, `\s` may be used inside
`\d` `\w` `\s`	A digit (0–9); a letter, digit or `_` (including non-English letters and CJK); white space (including the no-break and ideographic spaces)
`\D` `\W` `\S`	The opposite of each
`^` `$`	Start and end of a line
`*` `+` `?`	The item before, repeated 0 or more, 1 or more, or 0 or 1 times (greedy)
`\t`	A tab
`\` before punctuation	That character itself, for example `\.` or `\[`

There are no groups or alternation (`|`). A backslash before any other letter or digit is reported as "Bad regular expression", so those can be given meanings later. A pattern that would take too long (more than 100 million backtracking steps, as `a*a*a*a*b` can on a long line of a's) stops with "Search too complex - stopped" instead of hanging.

Replace. After "Replace:" and "With:", each match from the cursor to the end of the file is shown highlighted with the question "Replace? (Y)es (N)o (A)ll Esc stops". **Y** replaces it, **N** skips it, **A** replaces it and all the rest without asking, and any other key stops. The replacement is plain text, even in regex mode. Afterwards the prompt line says how many were replaced, for example "3 replaced". Tabs can be typed in all these prompts.

Cursor and editing keys

Key	Action
Arrows	Move one character or line. Left at the start of a line goes to the end of the line above
Home / End	Start or end of the line
PgUp / PgDn	Move by one screenful minus 5 lines
Ctrl+PgUp / Ctrl+PgDn	Top or bottom of the file
Ctrl+Left / Ctrl+Right	Scroll the view 8 columns left or right. If the cursor would leave the window, it moves to the nearest column still showing

Key	Action
Ins	Switch between insert and overstrike
Del / Backspace	Delete the character at the cursor or before it
Enter	Split the line (inserts CR/LF)

A diamond (♦) in inverse video at the right edge means the line continues past the window. Resizing the console window redraws the editor at the new size.

Sideways scrolling. The view follows the cursor. Moving past the right edge scrolls the view right, in steps of 8 columns, and moving back past the left edge scrolls it left again, so long lines can be edited without Ctrl+Left and Ctrl+Right. Up and Down keep to the same column within the line, scrolling if that column is off the screen.

Wide characters. Chinese, Japanese and Korean characters, fullwidth forms and emoji take two columns, as the console draws them. Characters above U+FFFF (most emoji, rarer CJK) are sent to the console as surrogate pairs. A wide character that would be cut by the right edge or the left margin shows as a blank. A narrow character above U+FFFF (for example Gothic or musical symbols) can't fit in one console cell and is shown as □; it is still stored and saved correctly.

TED draws on a console screen buffer of its own. When you quit, the console switches back to its original buffer, so whatever was on screen before you started TED (your command history and output) reappears unchanged, with the exit banner below it.

Saving

F10 (save and keep editing) and F7 or Ctrl+Q (save and quit) use the same steps:

1. TED writes the text to `name.$$$`.
2. For a new file, `name.$$$` is renamed to `name`.
3. For an existing file, any old `name.BAK` is deleted, and Windows' `ReplaceFile` swaps the new file in and turns the old one into `name.BAK` in one step. The saved file keeps the original's security settings (ACLs), attributes such as hidden or compressed, alternate data streams and creation time.
4. If the file has **hard links** (other names for the same file), replacing it would cut them off. Instead TED copies the file to `name.BAK` and then writes the new text into the file itself, so every name sees the new text and the file keeps its identity and metadata. The same in-place save is used when a drive doesn't support `ReplaceFile`, as on some network shares.
5. If `ReplaceFile` fails after it has already moved the original aside, TED renames `name.$$$` into place, and if even that fails, writes the text straight to `name`.

Before writing anything, TED checks for problems. Each shows a message on the prompt line, waits for a key, and returns to the "Save as:" prompt:

Message	Meaning
File is read-only - choose another name	The file has the read-only attribute; it is never overwritten

Message	Meaning
That is a folder - choose another name	The name is a folder
That folder doesn't exist - choose another name	The path is wrong
The .BAK file is read-only - choose another name	The old backup can't be replaced
File changed on disk by another program - overwrite? (Y/N)	Another program has changed the file since TED loaded or last saved it, or replaced it with a new file of the same name. **Y** saves anyway (the other program's version becomes `name.BAK`); any other key returns to the prompt

Later problems have messages too: "Can't replace the .BAK file" (the old backup is open in another program; the original is untouched), "Can't make the .BAK copy", "Can't create the file" and "Not enough memory to save". In the rare case that the in-place write fails, the message is "Couldn't save it - your text is in name.$$$", and the previous version is in `name.BAK`. An empty name just beeps.

How changes are spotted. TED notes the file's identity (volume and file ID, so the same file reached by another path or a hard link counts), its last-write time and size, and, for files up to 256 MB, a 64-bit FNV-1a hash of its contents. Before saving it compares them again. A different time or size means a change. If they match, the hash is checked too, which catches a change made within the time stamp's resolution (2 seconds on FAT drives) that kept the size. Saving to a different existing file with Save as gives no warning.

Because every save makes a backup, `name.BAK` always holds the version before the most recent save, not necessarily the file as it was when you opened it. If you change the name at the "Save as:" prompt, later saves use the new name.

---

3. How it works

3.1 From DOS to Win32

Original (DOS, 1988)	TED64 2.0
One 64 KB `.COM` segment plus `ORG 100h`	Flat 64-bit model with `.data`, `.data?` and `.code` sections
Flat 64 KB text buffer with 16-bit offsets	Gap buffer of up to 4 GB, positions held as 64-bit pointers
Direct writes to video memory at `B800:0`	An in-memory `CHAR_INFO` array copied with `WriteConsoleOutputW` to a screen buffer of its own
Code page 437 bytes on screen	UTF-8 text decoded and shown as UTF-16, wide characters in two cells
`INT 16h` keyboard	`ReadConsoleInputW`, translated back to BIOS-style ASCII/scan-code pairs plus a Unicode character

Original (DOS, 1988)	TED64 2.0
`INT 21h` file and memory services	`CreateFileW`, `ReadFile`, `WriteFile`, `ReplaceFileW`, `CopyFileW`, `MoveFileW`, `GetFileInformationByHandle`, `VirtualAlloc` and others
Command tail at `80h`	`GetCommandLineW`, parsed into UTF-8 arguments
`INT 23h` / `INT 24h` handlers	Raw console input mode, `SetConsoleCtrlHandler` and `SetErrorMode`
`INT 17h` printer (F3 PRINT)	Removed. F3 is now COPY
Fixed 80×25 screen (EGA/VGA rows detected once)	Any window size, re-checked every ¼ second and on every key

3.2 Program flow

```
main
 ├─ Console setup: handles, window size, colours, raw input mode,
 │   CTRL_HANDLER installed (Ctrl+Break ignored; recovery on close)
 ├─ Reserve the file, paste and conversion buffers and the undo/redo
 │   histories (smaller if address space is limited); screen image
 ├─ Parse the command line (GetCommandLineW → NEXT_ARG): /T:n, filename
 ├─ Open the file with CreateFileW (a missing file is a new one; any other
 │   failure is reported and TED exits with code 1)
 ├─ Load: read in 1 GB chunks, convert LF-only line endings, strip a
 │   trailing Ctrl-Z; RECORD_LOADED notes the file's identity, time, size
 │   and hash
 ├─ USE_OWN_SCREEN, REDO_PROMPT, and a notice if a .SAV file is waiting
 └─ READ_A_KEY loop
      ├─ After a resize: re-locate the cursor (APPLY_RESIZE)
      ├─ Redraw the prompt line if the "*" changed
      ├─ Update the marked range if marking is on
      ├─ Redraw the whole screen (DISPLAY_SCREEN) or the current line
      │   (DISPLAY_CURRENT); a full redraw is skipped while keys are waiting
      ├─ GET_KEY: check the window size, flush the screen, wait for a key
      │   (the one point where the control handler may save the text)
      ├─ CLASSIFY: typing, erasing or another command (for undo steps)
      └─ Dispatch:
           non-ASCII character → INSERT_CHAR (stored as UTF-8)
           printable / Enter    → INSERT_KEY
           Backspace            → LEFT + DEL_CHAR
           Ctrl+C / F / G / L   → COPY / FIND_CMD / GOTO_LINE / FIND_AGAIN
           Ctrl+H / Z / Y       → REPLACE_CMD / UNDO / REDO
           Ctrl+X / V / Q       → CUT / PASTE / EXIT
           Alt+F, Ctrl+arrows   → FIND_CMD, TOP / BOTTOM / SH_LEFT / SH_RIGHT
           scan 3Bh–58h (F1–F12) → DISPATCH_TABLE[scan − 3Bh]

On exit (F1, F7, or an error): FINISH → QUIT, which switches back to the
user's screen buffer, prints the banner, restores the keyboard mode and ends.
```

3.3 The gap buffer

The text lives in one buffer, split around an unused **gap**:



An insert or delete first moves the gap to the edit point (`MOVE_GAP`), which copies only the text between the old and new edit points. Typing then simply fills the gap. When the gap runs out,

`GROW_GAP` widens it by the amount needed plus an eighth of the file plus 64 KB, so growing is rare. Moves use a `MEMMOVE` routine that copies backwards in 8-byte words, because `REP MOVSB` is slow going backwards.

Positions. Every stored position is a pointer that never points *inside* the gap. The position right at the gap is always written as `GAP_END`. Because of this:

- Positions compare correctly with ordinary `CMP` instructions.
- Scanning code steps with the `NEXTP` and `PREVP` macros, which jump across the gap.
- `FIND_EOL` and `FIND_CR` run `REPNE SCASB` on the two halves in turn.
- The start of the text is `TEXT_BEG` (normally `R15`, or `GAP_END` when the gap is at the very start).
- Byte counts across the gap use `DIST`.

Keeping positions valid. The stored positions (`CURSOR`, `TOP_OF_SCREEN`, `MARK_START`, `MARK_END`, `MARK_HOME`) are listed together at `PTR_VARS`. `FIX_RANGE` and `FIX_SET` adjust them whenever text moves. As a result, a marked range stays attached to the text even if the text is edited while marking.

Saving. The gap is first moved to the end, so the text is one contiguous block that can be written directly.

Buffer	Pointer	What it holds
File (edit) buffer	`R15` (base, fixed for the whole run)	The gap buffer, always with CR/LF line endings
Paste buffer	`PASTE_SEG`	Text from Cut or Copy, or taken from the clipboard
Conversion buffer	`CONV_BUF`	The raw file during load (released afterwards), and the LF-only copy during save
Screen image	`SCREEN_BUF`	`CHAR_INFO` cells, 4 bytes each
Line buffer	`LINE_BUFFER`	Text deleted by F8/F9
Undo and redo histories	`HIST_UNDO`, `HIST_REDO`	Change records (see 3.6)

The three large buffers and the two histories reserve address space with `VirtualAlloc(MEM_RESERVE)`. `COMMIT_BUF` then commits real memory only as each buffer grows, keeping at least 64 KB spare past the end and growing by about an eighth at a time.

Each buffer has a two-quadword descriptor: bytes committed and bytes reserved (`EDIT_COMMIT`, `PASTE_COMMIT`, `CONV_COMMIT`, and inside `HIST_UNDO` and `HIST_REDO`). `RESERVE_BUF` asks for 5 GB first (256 MB for a history). If Windows refuses, as it can for a program run under a job object or another address-space limit, it halves the request until one succeeds, giving up below 64 MB. `COMMIT_BUF` and `GROW_GAP` respect each buffer's actual reservation. A request larger than the reservation fails cleanly: "Not enough memory" at load, or a beep while editing. If the histories can't be reserved at all, TED still runs, without undo.

3.4 Unicode

Text is treated as UTF-8:

- **`DECODE`** reads one character, crossing the gap if need be. It rejects overlong forms, surrogates and values above U+10FFFF. A byte that doesn't start a valid sequence counts as one Latin-1 character, so files in other encodings still open, display and save byte for byte.
- **`PREV_CHAR`** steps back over a whole character, and **`ENCODE_UTF8`** turns a typed character into bytes.
- **`GLYPH`** turns control characters into their code page 437 pictures (a lone LF shows as ◙) and C1 controls into □.
- **`CP_WIDTH`** gives a character's width, 2 for the ranges in `WIDE_TABLE` (Hangul Jamo, CJK radicals to Yi, Hangul syllables, CJK compatibility, fullwidth forms, the emoji blocks and the supplementary ideographic planes), otherwise 1.
- **`DRAW_GLYPH`** writes one character to the screen image: one cell, or two marked `COMMON_LVB_LEADING_BYTE` and `COMMON_LVB_TRAILING_BYTE`, holding the character twice or, above U+FFFF, its two surrogates. The prompt line, messages and the text all go through it.

Cursor movement, Delete, Backspace, overstrike and the column count all work in whole characters. `GET_KEY` reads UTF-16 from `ReadConsoleInputW`, combining surrogate pairs. Characters outside ASCII are reported in `KEY_CHAR` and inserted as UTF-8.

File names. `NEXT_ARG` turns the UTF-16 command line into UTF-8 arguments (with the usual quoting rules), and names are kept as UTF-8 throughout. `TO_WIDE` converts a name back to UTF-16 just before each `…W` file call. The exit banner and error messages are written with `WriteConsoleW` (surrogate pairs included), or as raw bytes if output is redirected.

3.5 Screen, columns and tabs

`CUR_POSN` packs the screen row (high 16 bits) and column (low 16 bits). Routines pass positions in `EDX` the same way. Columns within a line (`SAVE_COLUMN`, `LEFT_MARGIN`, `MARGIN_COUNT`, `LINE_COL`) are 32-bit.

- **`DISPLAY_LINE`** walks a line, skipping `LEFT_MARGIN` columns (`PUT_CHAR` shows half of a wide character there as a blank), and expands tabs to the next stop from **`NEXT_TAB`**, which uses `TAB_WIDTH`.
- **`CURSOR_COL`** works out the cursor's column within its line (`LINE_COL`) and the width of the character under it (`CUR_WIDTH`). Unless `HSCROLL_OFF` is set, it then moves `LEFT_MARGIN`, in steps of 8, so the whole character is on screen, and marks the screen for a redraw. Every cursor movement ends here, so the view always follows the cursor.
- **`SAVE_COLUMN`** is the column within the line that Up and Down aim for. **`SHIFT_RIGHT`** moves along a line to the character that covers that column.
- **`SHIFTED`** (after Ctrl+Left/Right) sets `HSCROLL_OFF` so the view stays where the user put it, and moves the cursor to the nearest visible column if it went off screen. `HSCROLL_OFF` is cleared at the next key.

`POSITION` turns a row/column into a cell address. `FLUSH_SCREEN` copies the image to the console in bands of about 15,000 cells, so no single `WriteConsoleOutputW` call is huge. This happens only in `GET_KEY`, just before waiting for a key.

3.6 Undo and redo

Every change goes through `INSERT_BYTES` or `DELETE_RANGE`, and both call **`LOG_EDIT`**. It pushes a record onto the undo history: the text offset, the length, a serial number, a group number and the type, plus, for a deletion, the deleted bytes. The 40-byte footer goes last, so the top record can be read from the end of the stack.

- **Groups.** `CLASSIFY` looks at each key before it is handled. Any key other than typing or erasing starts a new group, so one command is one undo step. Typing continues a group until a space or Enter has been typed, and Delete and Backspace presses continue one too. Consecutive typing is merged into one record, and so are Delete presses at the same place.
- **Undo** (`UNDO_REDO`) takes the records of the top group off the undo stack and applies the opposite change (deleting inserted text, or putting deleted text back). That change is itself logged, onto the redo stack, with the same serial and group. **Redo** does the same from the redo stack back onto the undo stack. A new ordinary change clears the redo stack.
- **The * flag.** The text's state is the serial number of the top undo record (or `EMPTY_ID` when the history is empty). Saving records it in `SAVED_SERIAL`, and `UPDATE_MODIFIED` shows the * whenever the two differ, so undoing back to the saved text clears it.
- **Limits.** If a history can't hold a record, `LOG_EDIT` empties it, and `EMPTY_ID` takes a fresh number so the text still counts as changed.

3.7 Find, regular expressions and Replace

`COMPILE` turns the search string into a list of 16-byte nodes: a type (character, any, set, negated set, `\d`/`\w`/`\s` and their opposites, start or end of line), a value, a repeat (`*`, `+`, `?` or once), and for a set the number of ranges in `CLASS_ITEMS`. Plain text compiles to character nodes. A character node holds the lower-case form from **`FOLD`**, which covers A–Z, Latin-1, Latin Extended-A, Greek and Cyrillic. Sets are tested with the character as it is, in lower case and in upper case (**`UPCASE`**).

`SEARCH` tries each position in turn. When every match must begin with a particular ASCII character, it skips ahead byte by byte to the next candidate, which is safe because an ASCII byte is never part of a longer UTF-8 sequence. **`MATCH`** is a backtracking matcher: a repeat first takes as many characters as it can (`ONE` matches one, never a line end), then gives them back one at a time with `PREV_CHAR`, so no list of positions is needed and the recursion is only as deep as the pattern. An empty match is rejected. `MT_STEP` counts backtracking steps and stops the search after `STEP_LIMIT`.

`REPLACE_CMD` highlights each match by setting `MARK_START` and `MARK_END` (marking itself stays off), redraws with **`SHOW_CURSOR`** (which keeps the screen still if the match is already showing), and asks. A replacement is a `DELETE_RANGE` and an `INSERT_BYTES`, so it is logged like any other change and the next search starts after the inserted text.

3.8 The clipboard

`CLIP_PUT` runs after every Cut and Copy. It converts the paste buffer to UTF-16 (`DECODE_STR`, so stray bytes become their Latin-1 characters) in memory from `GlobalAlloc`, and hands it over with `SetClipboardData(CF_UNICODETEXT)`. The clipboard needs an owner window, so a hidden `STATIC` window is created for the moment and destroyed straight after; a window kept open would be sent

clipboard messages that a console program never reads. `CLIP_PUT` notes `GetClipboardSequenceNumber` afterwards. `CLIP_GET`, at the start of every Paste, compares that number: if the clipboard still holds TED's own text, the paste buffer is used as it is; otherwise the clipboard's text is converted into it, with lone CRs and LFs made into CR/LF.

3.9 Closing the window

Windows calls `CTRL_HANDLER` on a thread of its own. For Ctrl+C and Ctrl+Break it returns TRUE (handled). For `CTRL_CLOSE_EVENT`, `CTRL_LOGOFF_EVENT` and `CTRL_SHUTDOWN_EVENT` it saves the text first:

1. It sets `HANDLER_ACTIVE` with `XCHG`, which acts as a full memory barrier.
2. It waits, up to 4 seconds, for `BUSY` to be 0. The main thread clears `BUSY` when `GET_KEY` starts waiting for a key (the text is never half changed then), and on leaving `GET_KEY` sets it again with `XCHG` and checks `HANDLER_ACTIVE`. If the handler has started, the main thread stops there for good. Whichever way the two threads meet, they never use the text at the same time.
3. If there are unsaved changes, `WRITE_RECOVERY` writes the two halves of the gap buffer to `name.SAV` (`MAKE_SAV_NAME`). It uses `BUF_BASE`, since `R15` belongs to the main thread.
4. It returns FALSE, and Windows ends the program.

3.10 The editor's own screen buffer

`USE_OWN_SCREEN` runs just before the first draw. It:

1. Creates a buffer with `CreateConsoleScreenBuffer`.
2. Sizes it to the largest window the console allows, using `GetLargestConsoleWindowSize` but never smaller than the current window, so the window can still be enlarged.
3. Makes it active with `SetConsoleActiveScreenBuffer`.
4. Restores the window to its previous size with `SetConsoleWindowInfo`.

From then on all drawing goes to that buffer (`H_OUT`), and the console's own buffer is kept in `H_ORIG`. `QUIT` switches back to `H_ORIG` before printing the banner. If any step fails, TED draws on the original buffer.

3.11 Window resizing

`GET_KEY` waits for input with `WaitForSingleObject` and a ¼-second timeout. Before each wait, `CHECK_RESIZE` re-reads the window with `GetConsoleScreenBufferInfo`. Polling catches resizes even in consoles that don't send buffer-size events.

On a change, `SET_GEOMETRY` updates the sizes, the text and prompt are redrawn straight away, and `GET_KEY` returns a dummy key (`RESIZE_KEY`). The main loop then runs `APPLY_RESIZE`, which keeps the cursor on screen and rebuilds the view around it with `LOCATE`. Prompts such as "Save as:" simply redraw at the new size.

3.12 File input/output and line endings

- **Open:** `CreateFileW` with `FILE_SHARE_READ | FILE_SHARE_WRITE`. If it fails, `GetLastError` decides what happens. `ERROR_FILE_NOT_FOUND` and `ERROR_PATH_NOT_FOUND` mean a new file. A sharing

violation, access denied (a folder gives this too, so `GetFileAttributesW` tells the two apart) or any other error is reported, and TED exits with code 1.
- **Load:** `GetFileSizeEx`, then read in 1 GB chunks (a single `ReadFile` call takes a 32-bit byte count).
- **Line endings:** one pass counts the line feeds that have no carriage return in front of them. A second pass copies the text into the edit buffer, adding a CR before each of those. If LF-only line breaks are in the majority, `LF_MODE` is set, and on save every CR/LF is written back as a bare LF. A file therefore keeps its original style.
- **Ctrl-Z:** a trailing 1Ah byte is stripped on load. It is only added back if `ADD_EOF_MARK = 1`.
- **Save:** `SAVE_FILE` checks the name, the `.BAK` file and the stamp, then `PREPARE_OUTPUT` gives the bytes to write (the text with the gap moved to the end, or an LF-only copy) and their hash. `WRITE_FILE` writes them in 1 GB chunks, to `name.$$$` and, for an in-place save, to the file itself (see "Saving" above).
- **Stamps:** `RECORD_LOADED` and `RECORD_SAVED` keep the volume serial number, file index, last-write time and size from `GetFileInformationByHandle`, the FNV-1a hash (`FNV_UPDATE`), and the name. `CHECK_STAMP` gets the same information for the file about to be replaced. If it is the same file and the time or size differs, or else `HASH_FILE` (reading it in 1 MB pieces) gives a different hash, it was changed. If it is a different file under the same name, it was replaced. Either way TED asks.

3.13 Program structure and calling conventions

The internal routines keep the original's register-based conventions:

- `RSI` and `RDI` are text or screen pointers, and `RCX` is a count.
- `EDX` is the row and column.
- `CF` signals failure or "not found".
- `R15` is the file-buffer base throughout. `R12` and `R13` are loop registers in the load and save code.

Frames and unwind data. Every procedure is a `PROC FRAME` that starts with `push rbp / .pushreg rbp / mov rbp, rsp / .setframe rbp, 0 / .endprolog` and leaves through `lea rsp, [rbp] / pop rbp` followed by `RET` or a `JMP` to the start of another procedure. Conditional jumps to another procedure go through a short local stub that does the same. The linker builds the `.pdata` and `.xdata` tables from these directives, so a debugger, a crash dump or an exception can walk the stack through any TED routine, and the `RBP` chain always links the frames.

Every Windows API call goes through the **`CALLWIN` macro**, which aligns the stack to 16 bytes and reserves shadow space plus room for up to three stack arguments. It keeps the old stack pointer in `RBX`, so `RBP` is still the calling procedure's frame pointer during the call.

Structure. Every procedure is entered only at its start, and every one ends in `RET`, `JMP` or `ExitProcess`, so none falls through into the next. `DISPATCH_TABLE` has 30 entries covering scan codes 3Bh–58h: F1–F10 (3Bh–44h), the cursor and keypad keys (47h–53h) and F12 (58h). Unused codes, including F11 (57h), point to `BAD_KEY`, which simply returns.

3.14 Procedure map (146 procedures)

Group	Procedures
Startup, main loop, closing	`main`, `CTRL_HANDLER`, `MAKE_SAV_NAME`, `WRITE_RECOVERY`, `WR_PART`, `BAD_KEY`, `USE_OWN_SCREEN`, `SET_GEOMETRY`, `APPLY_RESIZE`
Cursor movement	`LEFT`, `RIGHT`, `CURSOR_MOVED`, `UP`, `DOWN`, `SCROLLED`, `HOME`, `ENDD`, `PGUP`, `PGDN`, `PAGE_MOVE`, `TOP`, `BOTTOM`, `SH_LEFT`, `SH_RIGHT`, `SHIFTED`, `SHIFT_RIGHT`
Editing	`INSERT_KEY`, `INSERT_CHAR`, `DEL_CHAR`, `DEL_ONE_CHAR`, `DEL_EOL`, `DEL_L`, `DEL_TO_BUFFER`, `UDEL_L`, `INSERT_STRING`, `INSERT`
Undo and redo	`UNDO`, `REDO`, `UNDO_REDO`, `LOG_EDIT`, `HIST_ROOM`, `STATE_ID`, `UPDATE_MODIFIED`, `MARK_CLEAN`, `CLASSIFY`, `OFFSET_PTR`
Mark, cut, copy, paste, clipboard	`MARK`, `CUT`, `COPY`, `PASTE`, `PASTE_NO_ROOM`, `GROW_PASTE`, `CLIP_PUT`, `CLIP_GET`
Find, replace, go to line	`FIND_CMD`, `FIND_AGAIN`, `REPLACE_CMD`, `SHOW_CURSOR`, `UTOA`, `GOTO_LINE`, `GOTO_CURSOR`
Regular expressions	`COMPILE`, `CP_ESCAPE`, `SEARCH`, `MATCH`, `MT_STEP`, `ONE`, `IN_CLASS`, `IC_TEST`, `BUILTIN`, `FOLD`, `UPCASE`
Prompts and strings	`EDIT_FIELD`, `SHOW_MESSAGE`, `TTY_STRING`, `STRLEN`, `STRCPY_END`, `NEXT_ARG`, `TO_WIDE`
Exit and save	`ABORT`, `FINISH`, `QUIT`, `EXIT`, `SAVE`, `SAVE_FILE`, `PREPARE_OUTPUT`, `WRITE_FILE`, `CHG_EXTENSION`
Change detection	`RECORD_LOADED`, `RECORD_SAVED`, `STAMP_FROM_INFO`, `GET_INFO`, `CHECK_STAMP`, `STAMP_NAME_SET`, `STAMP_NAME_SAME`, `HASH_FILE`, `FNV_UPDATE`
Gap buffer	`MOVE_GAP`, `INSERT_BYTES`, `GROW_GAP`, `DELETE_RANGE`, `COPY_OUT`, `DIST`, `ADVANCE`, `FIX_RANGE`, `FIX_SET`, `UPDATE_BEG`, `MEMMOVE`
UTF-8	`DECODE`, `DECODE_STR`, `PREV_CHAR`, `ENCODE_UTF8`, `GLYPH`
Line and text scanning	`FIND_START`, `FIND_NEXT`, `FIND_PREVIOUS`, `FIND_CR`, `FIND_EOL`, `SKIP_CR_LF`, `LOCATE`, `CURSOR_COL`
Display	`DISPLAY_SCREEN`, `DISPLAY_BOTTOM`, `DISPLAY_ROWS`, `DISPLAY_CURRENT`, `DISPLAY_LINE`, `PUT_CHAR`, `DRAW_GLYPH`, `WRITE_CELL`, `CP_WIDTH`, `NEXT_TAB`, `REDO_PROMPT`, `ERASE_EOL`, `POSITION`, `WRITE_NORMAL`, `WRITE_INVERSE`
Windows wrappers	`SET_CURSOR`, `FLUSH_SCREEN`, `GET_KEY`, `KEY_READY`, `CHECK_KEY_EVENT`, `CHECK_RESIZE`, `BEEP`, `PRINT_STR`
Memory	`RESERVE_BUF`, `COMMIT_BUF`

4. Analysis

4.1 Strengths

- **Very small and self-contained.** A ~31 KB executable with no C runtime, importing only from `kernel32.dll` and `user32.dll`.

- **Fast on huge files.** With the gap buffer, typing 2,000 characters at the top of a 20 MB file takes 0.08 s, against 13.8 s for version 1.2. A 300 MB editing session (typing at top, middle and end, a search, and a save) completes in about 1.3 s.
- **Memory to match the file.** Address space is reserved up front and memory is committed on demand, so small files use little memory. Where address space is limited, TED reserves less and keeps working.
- **Undo you can rely on.** Every change, Cut, Paste and Replace included, can be undone and redone, and the * tells you whether the text matches the saved file.
- **Leaves the console as it found it.** TED's own screen buffer means your previous console contents come back when you quit.
- **Unicode throughout.** UTF-8 text displays and edits as whole characters, wide characters line up as in other console programs, and file names can be anything Windows allows. Invalid bytes are kept exactly, so non-UTF-8 files are not damaged.
- **Safe saves.** Writing to a temporary file first means a failed write never damages the original. The previous version is kept as `.BAK`, permissions, attributes, streams and hard links are kept, read-only files are refused, and saving warns before overwriting changes another program made.
- **Hard to lose work.** A file that exists but can't be read is reported instead of appearing empty, Ctrl+Break is ignored, and closing the window keeps unsaved text in a recovery file.
- **Stable positions.** Marks and the screen top stay attached to the text through edits.
- **Debuggable.** Unwind data and an unbroken `RBP` chain on every procedure.

4.2 Remaining limitations

Area	Issue
Character widths	Widths come from a fixed table of the main wide ranges. Combining marks and zero-width characters take a column of their own, and emoji sequences (flags, skin tones, joined emoji) show as their separate parts.
Narrow characters above U+FFFF	A console cell holds one UTF-16 unit, so these (Gothic, musical symbols, some historic scripts) are shown as □. They are stored and saved correctly.
Regular expressions	No groups, alternation, counted repeats (`{n,m}`), back-references or look-around, and no matches across line ends. The replacement is plain text. `\d` is ASCII digits only.
Case folding	Covers Latin (to Latin Extended-A), Greek and Cyrillic. Other scripts with case (Armenian, Georgian, Latin Extended-B and beyond) match exactly.
Undo history size	256 MB each for undo and redo. A bigger change (cutting several hundred MB, say) empties the history. Undo isn't kept after you quit.
Recovery timing	Windows allows about 5 seconds when the window closes. A recovery file for a very large file may not finish, and if TED is in the middle of a long command (a big Replace, say) the text is written after 4 seconds as it stands.
Change detection	Files larger than 256 MB are compared by time and size only. Saving with Save as onto a different existing file doesn't warn.
In-place saves	Used for hard-linked files and drives without `ReplaceFile`. If the write fails partway, the file itself is incomplete until saved again (the previous version is in `.BAK` and the new text in `name.$$$`).

4.3 Possible improvements

1. **Full Unicode width data**, including zero-width and combining characters and emoji sequences.
2. **More regular-expression features:** groups, alternation, `{n,m}` and `\1`-style replacement.
3. **Search in a selection** and a whole-word option.
4. **Line numbers** and a column indicator on the prompt line.
5. **Restoring from a `.SAV` file** with one key, instead of opening it separately.

---

5. Revision history

Version	Changes
1.0 / 1.1 (1988)	Original DOS `.COM` editor by Tom Kihlken, *PC Magazine*
1.2 64-bit	x64 Win32 console port. F3 PRINT replaced by COPY (also Ctrl-C). LF-only line endings supported. File limit raised from 64 KB to 64 GB with memory committed on demand and chunked I/O. Robustness fixes.
1.3 64-bit	Gap buffer. UTF-8 text with the `…W` console functions. Find (Ctrl+F / Alt+F, Ctrl+L) and Go to line (Ctrl+G). Live window resizing. Modified flag with a "*" indicator, so F1/F7 only ask when needed. 16/32-bit columns (windows up to 1024 columns, lines of any length). F8/F9 line buffer raised from 256 bytes to 64 KB. Fixes: stray ♦ on a line that exactly fills the window at the end of the file, stale mark offsets after edits (and a crash in 1.2), Ctrl+C at the "Save as:" prompt no longer adds a control character.
1.4 64-bit	F10 is now SAVE: save and carry on editing (Esc cancels; "Saved" confirms). UDEL L moves from F10 to F12; F11 is unused. F7 and F10 share one save routine, which now also checks the final rename. The prompt line leaves out its spaces in windows narrower than 88 columns.
1.5 64-bit	Draws on its own console screen buffer and restores the user's on exit. Saves through `ReplaceFile`, keeping permissions (ACLs), attributes, streams and creation time, with a rename fallback; read-only files are refused with a message. Buffers are reserved smaller when address space is limited. Procedures restructured so none is entered except at its start. Fixed a stack leak (inherited from 1988) on keys with no command. 64-bit constants written as literals.
1.6 64-bit	Files that exist but can't be opened (in use, access denied, a folder) are reported instead of being treated as new; files are opened allowing other programs to keep writing. A control handler ignores Ctrl+Break (1.2–1.5 only ignored Ctrl+C). Saving warns if another program changed the file. Read-only or undeletable `.BAK` files are reported instead of just beeping. Exit code 1 after errors.
2.0 64-bit	Wide (CJK, fullwidth, emoji) characters drawn two cells wide, and characters above U+FFFF as surrogate pairs. Unicode file names. Automatic sideways scrolling (and Left at a line start now goes up even when scrolled). Multi-level undo and redo of every change (F2/Ctrl+Z, Ctrl+Y), with the * cleared when the text matches the saved file. Find with Latin/Greek/Cyrillic case folding and regular expressions (Ctrl+R), and Replace (Ctrl+H). Tab width option `/T:n`. Hard links kept by saving in place, also used where `ReplaceFile` isn't supported. Recovery file on closing the window, logoff or shutdown. Change detection by file identity and content hash. Unwind data and `RBP` frames on every procedure. Largest file lowered from 64 GB to 4 GB. Ctrl+X (CUT), Ctrl+V (PASTE) and Ctrl+Q (EXIT) added alongside F5, F6 and F7. Cut, Copy and Paste use the Windows clipboard.

---

6. Testing

The source assembles with no errors or warnings under two independent MASM-compatible assemblers, JWasm and UASM (`-win64`). Their machine code is the same apart from the order of base and index registers in 13 addresses, which encode identical instructions. It links with `lld-link` into a PE32+ console executable.

For behaviour testing, the same source was assembled for Linux and linked with a test harness. The harness implements every Windows function TED64 uses, with the Win64 calling convention, plus a fake console that takes scripted keys and records the screen. With it:

- **Differential testing against 1.2:** about 4,200 random editing sessions were run, each on a random file and with up to 300 keystrokes. They covered typing, deleting, moving, paging, marking, cut, copy, paste, undo, F8/F9 and undelete line, horizontal scrolling and tabs. Each session was run through both 1.2 and 1.3, and the saved files and screens were compared. Every difference was traced to an intended 1.3 change or a 1.2 bug (the 256-byte line buffer, stale mark offsets, the stray ♦, the tab and margin column rules, and one 1.2 crash).
- **Robustness testing of 1.3:** 1,000 random sessions with UTF-8 and invalid bytes, tabs, LF-only files, long lines, typed Unicode, Find, Go to line and window resizes. Every session ran to completion, made its backup and saved its final edit.
- **Feature checks:** UTF-8 display, movement, typing and the byte-exact round trip; modified flag with F1 and F7; Find, wrap-around, "Not found" and Alt+F; Go to line, including past the end and line 0; growing and shrinking the window; lines over 1,000 characters; a 300-column window; LF-only round trip; backups; type-ahead.

For 1.4, the save flow was checked directly. That covered saving and editing on (with the `*` clearing and returning), Escape, repeated saves with the gap mid-file, LF-only files saved twice, Save-as to a new name, an invalid path, F12 undelete (and F11 doing nothing), and the prompt line at 100, 88, 80 and 40 columns. The differential and robustness runs were repeated (400 + 400 sessions), with mid-session saves added to the robustness run, and found no regressions.

For 1.5:

- **Against 1.4:** about 1,300 random sessions (ASCII with tabs and horizontal scrolling, and Unicode with Find, Go to line and resizes). Every screen, message and saved file was identical to 1.4, apart from the intended screen buffer change.
- **Wider runs:** 400 more robustness sessions and 300 more comparisons with 1.2, all clean.
- **Direct checks:**
- The user's console text survives editing.
- The fallback when no screen buffer can be created.
- Saved files keep their permissions (640, where 1.4 left 644).
- Read-only files are refused with the message.
- New files save correctly.
- The rename fallback when `ReplaceFile` fails.
- Three F10 saves in a row.
- Reservations under 200 MB and 100 MB address limits.
- 90,000 presses of an unassigned key on a 512 KB stack: 1.4 crashed, and 1.5 finished and saved.

For 1.6:

- **Against 1.5:** about 700 random sessions. Screens, messages and saved files were identical to 1.5 in ordinary editing.
- **Wider runs:** 300 more robustness sessions and 200 more comparisons with 1.2, all clean.
- **Direct checks:**
- A locked file, an access-denied file and a folder each give their message and exit code 1, and the real file is left untouched. With 1.5, each opened as an empty buffer, and the locked and access-denied files were replaced by the next save.
- New files and missing folders still open as new, with exit code 0 on quit.
- Ctrl+Break during editing is ignored, and the work saves. (1.5, with no handler, was ended immediately.)
- Another program's change is detected before saving, both after loading and after an earlier save of our own. **N** returns to the prompt, and **Y** saves with their version kept as `.BAK`.
- No warning appears for repeated saves of our own or for Save-as to a different name.
- A read-only `.BAK` and an undeletable `.BAK` each give their message, and the original, the old backup and the folder (no temporary file left behind) are all unchanged.

For 2.0:

- **Against 1.6:** 1,300 random sessions on text that 1.6 also handles in the same way (lines that fit the window, no wide characters, no F2, no Ctrl+Left/Right). They covered typing, deleting, moving, paging, marking, cut, copy, paste, F8/F9/F12, tabs, LF-only files, invalid UTF-8, Find, Go to line, resizes and saves. Screens, messages and saved files were identical.
- **Undo and redo:** 1,300 random sessions (typing, all the deleting keys, cut, paste, F8/F9/F12, overstrike, undo and redo mixed in, Unicode included). In every one, undoing everything gave back the original file byte for byte without a *, and then redoing everything gave the edited file, with the * matching.
- **Regular expressions and Replace:** 3,900 random patterns (sets, ranges, negated sets, `\d \w \s \D \W \S`, `. * + ? ^ $`, escapes) and plain strings, on text mixing ASCII, accented Latin, Greek, Cyrillic, CJK and emoji. Replace-all with `#` was compared against Python's `re` module with `IGNORECASE`, line by line. All matched.
- **Wide characters, tabs and scrolling:** 1,800 random sessions with CJK, emoji, tabs of widths 1–13, long lines, window widths from 16 to 80 and Ctrl+Left/Right, with the screen checked after every key: no half-drawn wide character, no unpaired surrogate, and the cursor always inside the window.
- **Frames:** both assemblers give 146 unwind entries, one per procedure, each with an `RBP` frame (checked with `llvm-readobj --unwind`). In the test build, the harness walked the `RBP` chain at every console and file call and always found it intact, up to 7 frames deep.
- **Direct checks:**
- Unicode folder and file names open and save, also when typed at "Save as:". `/T:4`, `/T4`, `-t:4` work; `/T:0` and `/T:33` give the usage line and exit code 1.
- A hard-linked file keeps both links after saving, and the other name sees the new text (1.6 cut the link). Without `ReplaceFile`, the in-place save keeps the file's permissions. A failing in-place write reports where the text is.
- A change through a hard-link alias is detected. A change with the same size and time stamp is detected by the hash (1.6 missed it). A file replaced by a new one of the same name is detected. Our own repeated saves and Save as to a different file give no warning.

- Closing the window with unsaved changes writes `name.SAV` (or `TED.SAV` with no name) holding the edited text, and the next start reports it. With no changes, no file is written.
- Typing runs undo a word at a time, the * clears and returns with undo/redo around a save, and undo with nothing left beeps.
- Interactive Replace with Y, N and A, the count message, "Bad regular expression", and "Search too complex - stopped" after about a second on a pathological pattern.
- **Clipboard:** with a fake clipboard in the harness, Copy then Paste puts back the exact bytes (including invalid UTF-8); text "copied by another program" is pasted with emoji intact and its line ends made CR/LF; a busy clipboard falls back to TED's own copy; and 200 random sessions mixing Cut, Copy, Paste, Ctrl+X/C/V, undo and outside copies never left the clipboard open or used it wrongly. A console-style paste (the text arriving as keystrokes, with an LF after each Enter) inserts the text once and undoes in one step.

The test harness also needed one fix this round: it gave typed `[`, `\` and `]` the virtual-key codes of the Windows keys, which TED (correctly) ignores.

It has **not yet been assembled with Microsoft's own `ml64.exe` or run in a real Windows console**. Building it in Visual Studio and trying it on real files is the recommended first check, particularly for wide characters and emoji (which depend on the console and font: Windows Terminal handles them best), resizing, and closing the window with unsaved changes.